



Toronto Metropolitan University

Faculty of Engineering & Architectural Science

Department of Electrical, Computer and Biomedical Engineering

Course Title:	Computer Networks I
Course Number:	COE768 - 072
Semester/Year (e.g. F2016)	F2025

Instructor:	Dr. Truman Yang
Teaching Assistant (TA)	cugang@torontomu.ca

<i>Assignment/Lab Number:</i>	Lab 1
<i>Assignment/Lab Title:</i>	Study of the concepts of Layer Architecture

<i>Submission Date:</i>	September 25th, 2025
<i>Due Date:</i>	September 25th, 2025 (6:00 PM)

Student LAST Name	Student FIRST Name	Student Number	Section	Signature*
Maisam Abbas	Syed	501103255	07	

*By signing above you attest that you have contributed to this written lab report and confirm that all work you have contributed to this lab report is your own work. Any suspicion of copying or plagiarism in this work will result in an investigation of Academic Misconduct and may result in a "0" on the work, an "F" in the course, or possibly more severe penalties, as well as a Disciplinary Notice on your academic record under the Student Code of Academic Conduct, which can be found online at: <http://www.ryerson.ca/senate/current/pol60.pdf>

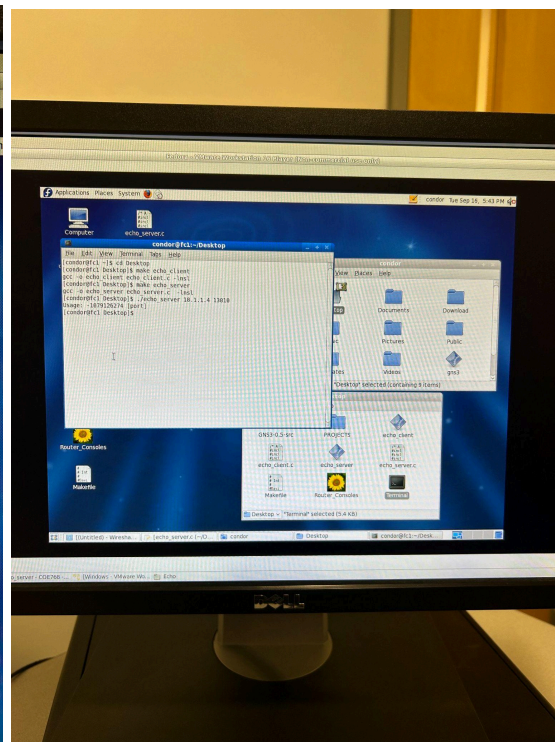
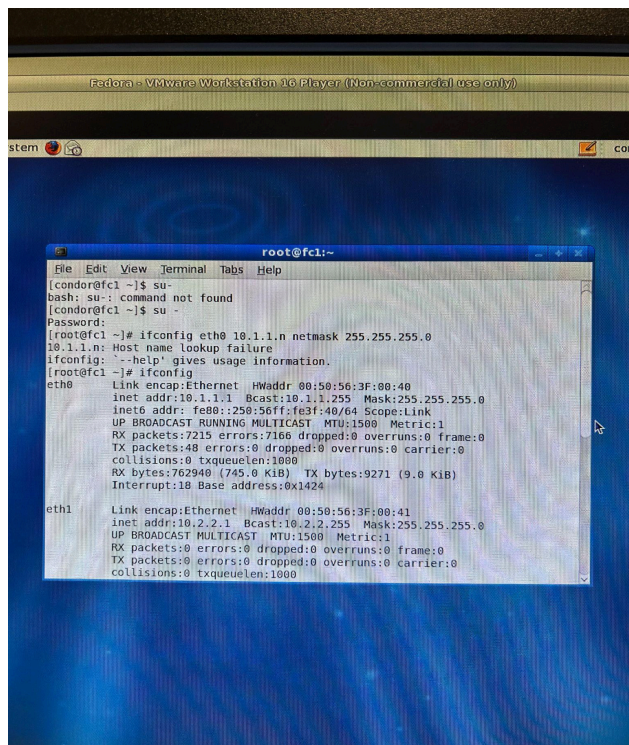
Table of Contents

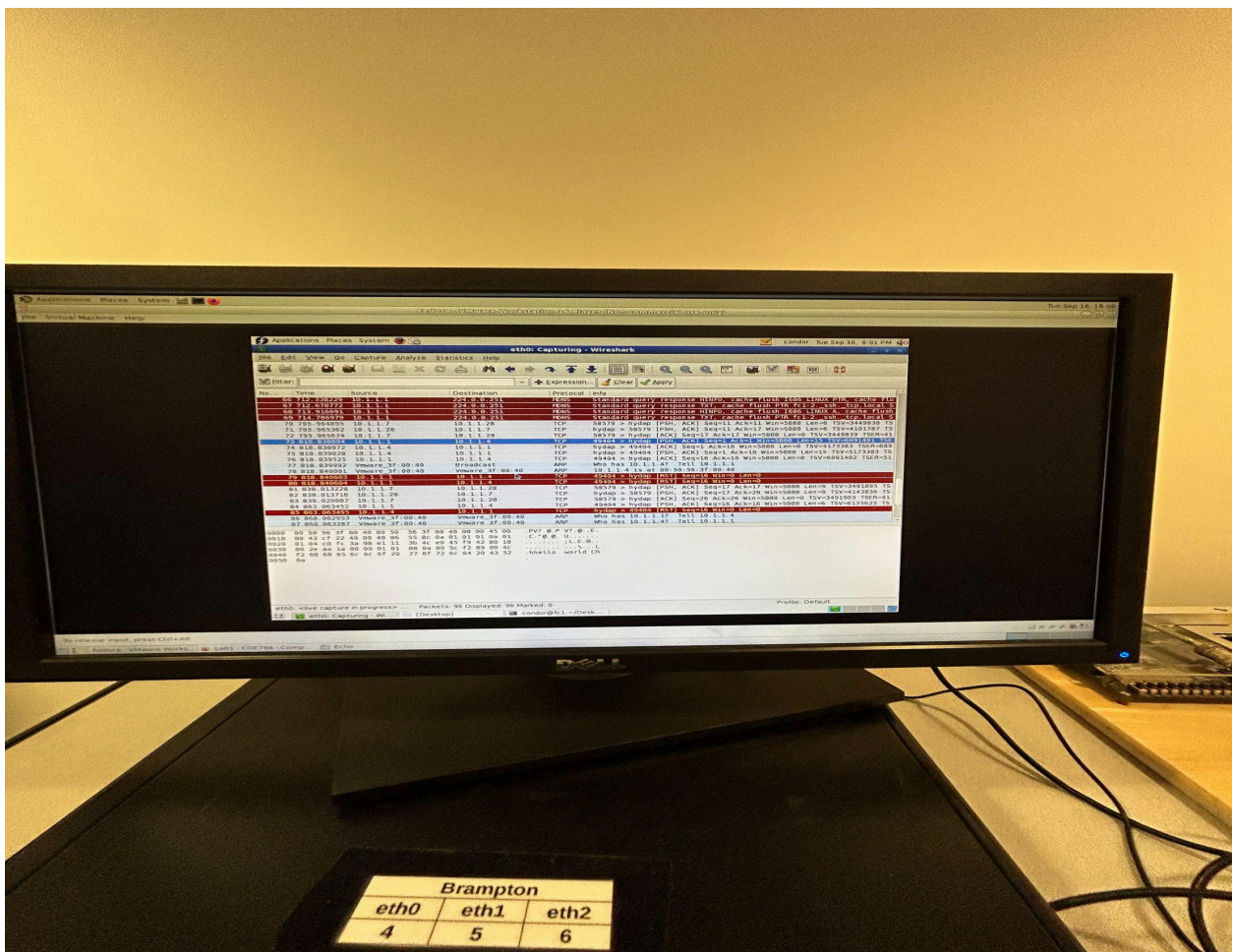
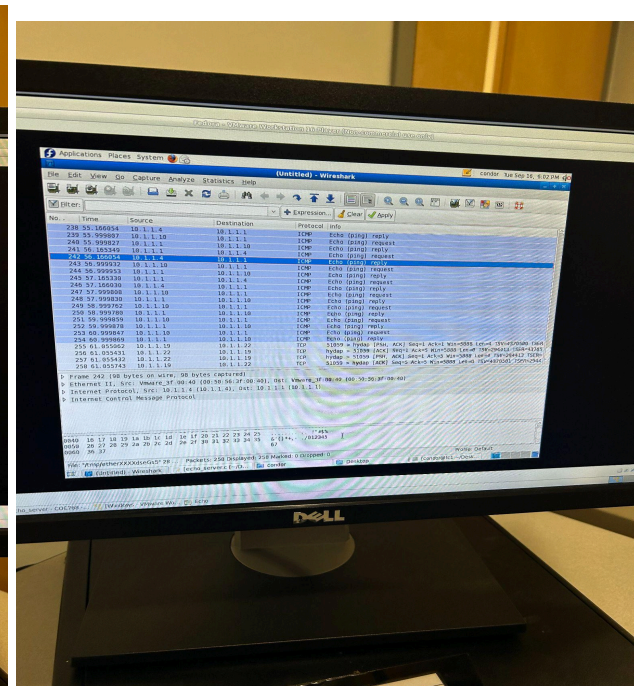
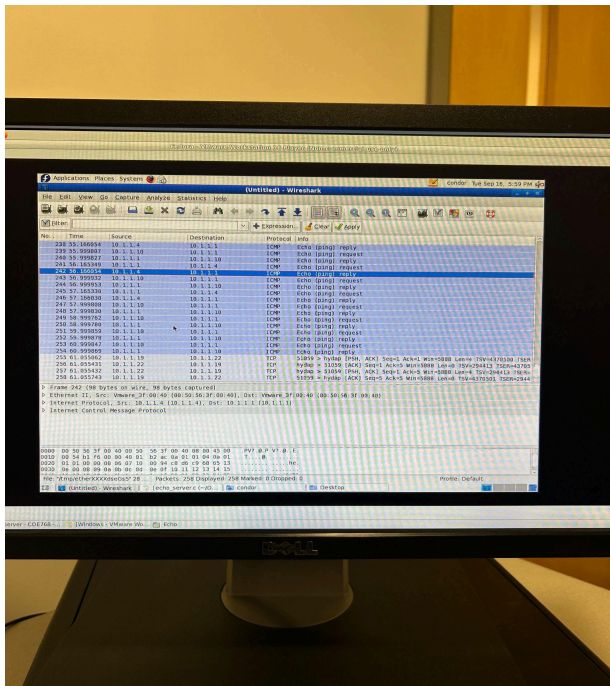
Objectives.....	3
lab Questions.....	10

Objectives

This lab introduces the Keil uVision IDE and some of its features. More specifically, the architecture of the ARM Cortex M3. The coding basics of configuring the LEDs and LCDs of the ARM Cortex M3 and its NXP LPC1768 microcontroller will be simulated, debugged, and analyzed, including how to run a simple program on the MCB1700 dev board. This lab will allow engineers to become familiar with the uVision environment, its simulation capabilities, and the tools needed to assess various CPU performance factors. As the majority of embedded systems use ARM processors for low-power consumption and competitive performance, designers will find the skill sets obtained from this lab especially useful.

- o Traffic analysis using Wireshark
- o Study of network encapsulation





Lab Questions of part (e)

- **3A:** At the application level, only two packets are exchanged between the client and the server. However, you will find that 8 or more TCP packets are actually generated. TCP is a connection-oriented protocol, that is, a logical connection must be established between two communication entities before data transfer (somewhat similar to the telephone connection). With this understanding, find out the functions of those extra TCP packets.

Functions of Extra TCP Packets: The extra TCP packets generated during the communication between the client and server serve several functions related to establishing and terminating a connection, as well as acknowledging data transfer. TCP is a connection-oriented protocol, which means a logical connection must be established before data can be transferred, similar to a telephone connection. The additional packets beyond the two at the application level are for:

- ➔ Connection establishment: This process is often a three-way handshake involving SYN (synchronize) and ACK (acknowledge) packets.
- ➔ Data transfer: This includes the packets carrying the actual message from the client to the server and the response from the server back to the client.
- ➔ Packet acknowledgment: TCP uses ACKs to confirm the successful reception of packets.
- ➔ Connection termination: This involves a separate handshake to gracefully close the connection after data transfer is complete.

-
- **3B:** From the Wireshark window, locate the frame that contains the message sent from the client to the server. The frame is displayed in hexadecimal. Record the content of the frame and indicate the locations of the following parameters:
 - The Ethernet addresses of the client and server VMs;
 - The Ethernet type;
 - The IP addresses of the client and the server;
 - The protocol number;
 - The source and destination port numbers.

Analyzing a TCP Frame in Wireshark: To analyze a frame sent from the client to the server, you would use Wireshark to locate the frame containing the message. The content of the frame is displayed in hexadecimal format. You'd need to identify the locations of various parameters within this hexadecimal data to understand the packet's structure. The key parameters to look for are:

- ➔ Ethernet addresses: These identify the source and destination machines at the Ethernet layer (layer-2 addresses).
- ➔ Ethernet type: This field indicates that an IP packet is encapsulated by the Ethernet frame.
- ➔ IP addresses: These are the layer-3 addresses of the source and destination machines.

- Protocol number: This field in the IP header specifies the type of packet encapsulated by the IP datagram, in this case, a TCP packet. The IP protocol field will be different for TCP traffic than for ICMP traffic (like a ping).
 - Source and destination port numbers: These are part of the address for the network application process. The destination port number for the client-to-server packet will be the port number the server is listening on.
-

- **3C:** Identify the layers the packet had to go through before it was sent out.

Packet Layers: Before a packet is sent out, it must pass through several layers of the network architecture. The packet starts at the application layer (Echo service in this case), then moves down through the transport layer (TCP), the network layer (IP), and finally the link layer (Ethernet). At each layer, a new header is added to the packet, a process called encapsulation. For the Echo service, the packet would acquire headers in the following order:

- TCP header: Added at the transport layer, containing information like port numbers.
- IPv4 header: Added at the network layer, containing the source and destination IP addresses.
- Ethernet header: Added at the link layer, with source and destination Ethernet addresses.

The final structure of the packet before it is sent out is an Ethernet frame, which contains the Ethernet header, followed by the IP datagram (which includes the IPv4 header), and the TCP packet (with the TCP header and the Echo data).
